

Práctica de Bases de Datos Distribuidas

Caso Banco del Austro: guía paso a paso desde cero

Aplicaciones Distribuidas (ISR-701) · Unidad 2 · Semana 10–11

Ph.D. Gleiston Guerrero Ulloa
 Universidad Técnica Estatal de Quevedo
 Facultad de Ciencias de la Computación
 Carrera de Ingeniería de Software

Período académico 2026–2027 SPA

¿Para quién es esta guía?

Esta guía está escrita asumiendo que usted **ya sabe programar en Java y ya usó alguna vez PostgreSQL, Docker Desktop, IntelliJ y pgAdmin** pero es la primera vez que **va a construir una base de datos distribuida**. Por eso cada concepto de la Unidad 2 (sede, nodo, fragmento, réplica, contenedor, volumen) se explica en su propia caja antes de aparecer en el código, y cada archivo que va a crear tiene indicado con precisión *dónde guardarlo, con qué nombre, con qué extensión y con qué programa* escribirlo.

Al terminar la práctica habrá levantado dos motores PostgreSQL 16 independientes (uno para la oficina de Cuenca y otro para Quito) coordinados desde una aplicación Spring Boot que decide, para cada consulta, en qué motor buscar. Es el *prototipo didáctico* más pequeño que sigue evidenciando las tres propiedades centrales de una BDD: **fragmentación horizontal, transparencia de ubicación y autonomía local**.

Índice

1. Antes de empezar: glosario mínimo	3
2. Ambiente de trabajo: programas y dónde conseguirlos	4
2.1. Sistema operativo asumido	4
2.2. Programas obligatorios	4
2.3. Programa opcional pero recomendado	4
2.4. Verificación rápida del ambiente	5
3. El caso: Banco del Austro en modo prototipo	6
3.1. La historia	6
3.2. Del problema real al prototipo didáctico	6
3.3. Convención de prefijos	6
3.4. Estrategia de distribución por tabla	7
4. Crear la estructura de carpetas del proyecto	8
4.1. Decidir la carpeta raíz	8
4.2. Crear las carpetas desde <code>cmd.exe</code>	8
4.3. Estructura esperada	8
4.4. ¿Qué va a ir en cada carpeta?	8

5. Levantar los motores PostgreSQL con Docker	10
5.1. Repaso rápido de conceptos	10
5.2. Crear el archivo <code>docker-compose.yml</code>	10
5.3. Línea por línea: qué significa cada bloque	11
6. Crear las tablas y datos de prueba	13
6.1. Repaso rápido de conceptos	13
6.2. Crear los scripts del nodo Cuenca	13
6.3. Crear los scripts del nodo Quito	14
6.4. Estructura del proyecto en este punto	15
7. Levantar los contenedores por primera vez	16
8. Verificar los datos con pgAdmin (opción gráfica)	18
8.1. Configurar pgAdmin 4	18
8.2. Alternativa: panel Database de IntelliJ IDEA	19
9. Crear el proyecto Spring Boot	20
9.1. Repaso rápido de conceptos	20
9.2. Generar el proyecto	20
9.3. Estructura del proyecto que se genera	21
9.4. Cambiar <code>application.properties</code> por <code>application.yml</code>	21
10. Escribir el código Java de la aplicación	23
10.1. Organizar los paquetes	23
10.2. Clase de configuración de dos <code>DataSource</code>	23
10.3. Servicio de consulta distribuida	24
10.4. Controlador REST	25
10.5. Estructura del proyecto Java en este punto	26
11. Ejecutar y probar la aplicación	27
11.1. Levantar la aplicación Spring Boot	27
11.2. Probar los endpoints con Postman	27
11.3. Alternativa: probar con el navegador o con curl	28
12. Prueba de tolerancia a fallos	29
13. Detener y limpiar el ambiente	29
14. Cierre: qué se logró y qué queda pendiente	31
14.1. Propiedades verificadas	31
14.2. Qué queda pendiente para próximas prácticas	31
14.3. Ejercicios propuestos	31
14.4. Glosario final	32

1 Antes de empezar: glosario mínimo

En Unidad 1 usted trabajó con sockets, gRPC y relojes de Lamport [1]. En Unidad 2 va a cambiar el enfoque: ahora el *dato* es lo que está distribuido, no solamente los procesos que lo consultan. Antes de tocar Docker o IntelliJ es imprescindible fijar el vocabulario, porque los mismos términos significan cosas distintas según el contexto.

¿Qué es una “sede”??

Una **sede** (o *site*, *sitio*) es una ubicación física donde reside **una copia de un motor de base de datos con parte o todo el dato de la organización**. En este caso, Banco del Austro tiene cinco sedes reales: Cuenca, Quito, Guayaquil, Loja y Machala. En el prototipo didáctico se implementan solo dos (Cuenca y Quito) porque con dos ya se ilustran las mismas propiedades y los recursos del laboratorio se optimizan.

Cada sede corre su propio motor PostgreSQL, tiene su propio disco lógico y puede seguir operando aunque otra sede se caiga. Eso es lo que en la literatura de sistemas distribuidos se llama *autonomía local* [2].

¿Qué es un “nodo”??

Un **nodo** es cada motor de base de datos individual, considerado como participante del sistema distribuido. En esta práctica cada sede tendrá exactamente un nodo (una instancia de PostgreSQL) y por eso las palabras “sede” y “nodo” se usan de manera intercambiable. En sistemas más grandes una sede puede tener varios nodos por redundancia; en Unidad 3 se verá esa complicación.

¿Qué es un “fragmento”??

Un **fragmento** es un subconjunto de las filas (o de las columnas) de una tabla completa, que reside en un nodo específico. En la práctica se va a fragmentar *horizontalmente* la tabla **cuentas**: las cuentas cuya oficina de origen es Cuenca residirán físicamente solo en el nodo Cuenca, y las de Quito solo en el nodo Quito. Nadie almacena la tabla completa; la “tabla completa” es una entidad lógica que solo existe cuando se hace la unión de los fragmentos. La regla técnica es que los fragmentos deben ser **completos** (ninguna fila se pierde), **re-construibles** (uniendo los fragmentos se recupera la tabla original) y **disjuntos** (ninguna fila está en dos fragmentos al mismo tiempo) [2, cap. 3].

¿Qué es una “réplica”??

Una **réplica** es una copia *completa* de una tabla que se mantiene en más de un nodo, para acelerar las lecturas o tolerar la caída de un nodo. En esta práctica la tabla **clientes** se replica: los mismos clientes existen en Cuenca y en Quito, porque cualquier oficina debe poder identificar al titular de una cuenta aunque este pertenezca a otra sede.

La contrapartida es que las *escrituras* sobre una tabla replicada son más caras porque hay que actualizar todas las copias [3].

2 Ambiente de trabajo: programas y dónde conseguirlos

Esta sección lista los programas que va a usar, con qué sirve cada uno y dónde descargarlos oficialmente. Instale todos antes de continuar; los pasos posteriores asumen que ya están listos.

2.1 Sistema operativo asumido

La guía está redactada para **Windows 10 o Windows 11**. En Linux o macOS los conceptos son idénticos y la mayoría de comandos también; únicamente cambia la ruta de trabajo (`/home/usuario/uteq/` en lugar de `C:\uteq\`) y el terminal (`bash` o `zsh` en lugar de `cmd.exe`).

Todos los comandos de esta guía se ejecutan en la **Consola de comandos de Windows**, también llamada `cmd.exe` o simplemente “Símbolo del sistema”. **No use PowerShell**: algunos comandos con comillas se comportan distinto en PowerShell y le harán perder tiempo.

Para abrir `cmd.exe`: presione la tecla *Windows*, escriba `cmd` y pulse *Enter*. Debe aparecer una ventana negra con texto blanco donde el prompt termina en `>`.

2.2 Programas obligatorios

Estos cinco programas son indispensables. Instale cada uno usando el instalador oficial (evite descargar de sitios terceros).

Programa	Versión	Sirve para...
Docker Desktop	4.30 o superior	Ejecutar los dos motores PostgreSQL como <i>contenedores</i> , sin instalar PostgreSQL en su computadora. Descarga oficial: https://www.docker.com/products/docker-desktop/ .
JDK Temurin	21 (LTS)	Compilar y ejecutar la aplicación Java (Spring Boot) que consulta los dos motores. Se recomienda la distribución Eclipse Temurin: https://adoptium.net/temurin/releases/ .
IntelliJ IDEA	2024.1 Community o superior	Escribir el código Java, generar el proyecto Spring Boot y consultar visualmente las bases de datos desde su ventana <i>Database</i> . Descarga oficial: https://www.jetbrains.com/idea/download/ . La edición Community es gratuita y suficiente.
Postman	Versión actual	Enviar peticiones HTTP (<code>GET</code> , <code>POST</code> , etc.) a la aplicación Spring Boot para verificar que las consultas distribuidas responden correctamente. Descarga oficial: https://www.postman.com/downloads/ .
VS Code o Notepad++	Cualquiera	Editor de texto plano para crear los archivos <code>.yml</code> y <code>.sql</code> sin que Windows los “ayude” agregando caracteres invisibles. No use Word ni WordPad : agregan caracteres invisibles que rompen los scripts [4].

2.3 Programa opcional pero recomendado

Programa	Versión	Sirve para...
----------	---------	---------------

pgAdmin 4	8.x o superior	Ver el contenido de PostgreSQL con una interfaz gráfica separada de IntelliJ. Descarga: https://www.pgadmin.org/download/ . Útil si prefiere una herramienta dedicada a bases de datos; si le basta el panel Database de IntelliJ, puede omitirlo.
-----------	----------------	--

2.4 Verificación rápida del ambiente

Antes de continuar, abra `cmd.exe` y ejecute los siguientes comandos, uno por uno. Cada uno debe imprimir una versión; si alguno responde `'xxx'` **no se reconoce como un comando** entonces el programa correspondiente no está instalado o falta agregar su carpeta al `PATH`.

Listing 1: Verificación del ambiente en `cmd.exe`

```
docker --version
docker compose version
java -version
```

¿Cómo saber si salió bien?

Debe obtener respuestas similares a:

```
Docker version 24.x.x, build xxxxxxx
Docker Compose version v2.x.x
openjdk version "21.x.x" ...
```

Los números pueden ser distintos; lo importante es que las tres líneas devuelvan una versión.

Si algo falla...

Docker Desktop no arranca o está en “Docker starting...” eterno. Cierre Docker Desktop, verifique en el *Administrador de tareas* que no queden procesos **Docker*** corriendo, y vuelva a abrirlo como Administrador. En Windows 11, asegúrese de tener habilitado el *Windows Subsystem for Linux 2* (WSL 2): abra `cmd.exe` como Administrador y ejecute `wsl -install`.

java no se reconoce como comando. La variable `PATH` no incluye la carpeta `bin` del JDK. En Windows: *Configuración* → *Sistema* → *Acerca de* → *Configuración avanzada del sistema* → *Variables de entorno* → edite `PATH` y añada la ruta `C:\Program Files\Eclipse Adoptium\jdk-21...\bin`. Cierre `cmd.exe` y ábralo de nuevo.

3 El caso: Banco del Austro en modo prototipo

3.1 La historia

El Banco del Austro es una entidad financiera ecuatoriana con oficinas en cinco ciudades: Cuenca (sede central), Quito, Guayaquil, Loja y Machala. La dirección plantea el siguiente problema:

- Cada oficina consulta constantemente los saldos de *sus propios* clientes y esas consultas deben responder en menos de 200 ms.
- Las transferencias entre cuentas deben ser exactas: si el motor cae en mitad de una transferencia, o se transfirió correctamente o no se transfirió nada; no puede quedar la mitad. Esto es la propiedad *ACID* [5, cap. 10].
- Si la oficina de Loja pierde la conexión con Cuenca por unas horas, las oficinas restantes deben seguir operando con sus clientes locales.
- Los datos de clientes son de acceso compartido: cualquier oficina debe poder ver a cualquier titular.
- Los datos transaccionales pertenecen a la oficina donde se originaron y casi siempre se consultan localmente.

3.2 Del problema real al prototipo didáctico

En un sistema real de un banco se usarían cinco nodos, comunicación cifrada, replicación *active-active* y un producto como CockroachDB o PostgreSQL + Citus [6, 7]. Todo eso queda para prácticas posteriores. En ésta se reduce el problema a lo mínimo didáctico:

- Solo dos sedes: **Cuenca** y **Quito**.
- Solo tres tablas: **clientes**, **cuentas**, **transacciones**.
- Una única aplicación Spring Boot que decide, a partir del número de cuenta, en qué nodo consultar.
- Fragmentación horizontal explícita: cada nodo almacena solo los datos que “le tocan” según su ciudad de origen.

3.3 Convención de prefijos

Para que el enrutador de la aplicación pueda saber, sin ir a la base, en qué sede está cada cuenta, se adopta la siguiente convención de *codificación* del número de cuenta:

- Las cuentas de la oficina de Cuenca empiezan con 22. Ejemplo: 2201000001.
- Las cuentas de la oficina de Quito empiezan con 17. Ejemplo: 1701000001.

Nota técnica

Los prefijos 22 y 17 corresponden a los primeros dígitos de la cédula ecuatoriana asociada a las provincias del Azuay (Cuenca) y Pichincha (Quito). Se usan por su valor mnemotécnico; en producción el enrutamiento se implementaría con una tabla de directorio o con *hashing consistente*, no leyéndose del propio dato.

3.4 Estrategia de distribución por tabla

La siguiente tabla es el *plan de fragmentación* que se va a implementar. Es la decisión de diseño más importante de la práctica; el resto son pasos técnicos para llevarla a cabo.

Tabla	Estrategia	Explicación
clientes	Replicada	Copia completa en Cuenca y en Quito. Todos los titulares se ven desde cualquier oficina.
cuentas	Fragmentada horizontalmente por oficina	Las cuentas cuya oficina es CUENCA viven en el nodo Cuenca; las que tienen QUITO viven en el nodo Quito. Ninguna cuenta vive en los dos.
transacciones	Fragmentada horizontalmente por oficina	Cada transacción se registra donde se originó. Los movimientos “cross-office” se resolverán en una práctica posterior con commit en dos fases [5].

4 Crear la estructura de carpetas del proyecto

Antes de tocar Docker o IntelliJ, se prepara la carpeta donde van a vivir *todos* los archivos de esta práctica: los scripts SQL, el `docker-compose.yml` y luego el proyecto Java.

4.1 Decidir la carpeta raíz

Se recomienda crear la carpeta raíz en la raíz de la unidad C: para que la ruta sea corta y no colisione con espacios (los espacios en rutas causan errores en Docker):

```
C:\uteq\banco-austro
```

Si algo falla...

No use rutas con espacios ni con acentos, como C:\Users\Juan Pérez\Escritorio\Prácticas. Docker Desktop las acepta a veces, pero al montar volúmenes o carpetas dentro de contenedores puede fallar con mensajes crípticos. Cree la carpeta en C:\uteq\ y ya.

4.2 Crear las carpetas desde cmd.exe

Abra `cmd.exe` y ejecute:

Listing 2: Crear la raíz y las dos subcarpetas de scripts SQL

```
mkdir C:\uteq
mkdir C:\uteq\banco-austro
cd C:\uteq\banco-austro
mkdir sql-cuenca
mkdir sql-quito
```

Si la carpeta C:\uteq ya existe porque la usó en prácticas anteriores, ignore ese primer `mkdir`; Windows le dirá que ya existe y no pasa nada.

4.3 Estructura esperada

Al finalizar esta sección la carpeta debe verse así:

Listing 3: Estructura inicial del proyecto

```
C:\uteq\banco-austro\
+-- sql-cuenca\ (aquí irán los scripts SQL del nodo Cuenca)
+-- sql-quito\ (aquí irán los scripts SQL del nodo Quito)
```

Puede verificarlo desde el Explorador de Windows o desde `cmd.exe` con el comando `dir`:

```
cd C:\uteq\banco-austro
dir
```

Debe listar dos entradas de tipo <DIR>: `sql-cuenca` y `sql-quito`.

4.4 ¿Qué va a ir en cada carpeta?

Carpeta	Contendrá...
C:\uteq\banco-austro\	Un archivo llamado <code>docker-compose.yml</code> que le dirá a Docker cómo levantar los dos motores PostgreSQL. Además, más adelante, la carpeta del proyecto Java.

<code>sql-cuenca\</code>	Dos archivos <code>.sql</code> : uno con las tablas del nodo Cuenca y otro con los datos de prueba de Cuenca. PostgreSQL los ejecutará automáticamente al arrancar.
<code>sql-quito\</code>	Lo análogo para el nodo Quito.

5 Levantar los motores PostgreSQL con Docker

5.1 Repaso rápido de conceptos

¿Qué es una “imagen” de Docker??

Una **imagen** es una plantilla congelada de software: contiene el sistema operativo mínimo, PostgreSQL 16 preinstalado y toda su configuración por defecto. La imagen oficial se llama `postgres:16-alpine` y se descarga desde *Docker Hub*. La versión `alpine` es una variante pequeña (unos 250 MB) basada en la distribución Alpine Linux.

¿Qué es un “contenedor”??

Un **contenedor** es una instancia *en ejecución* de una imagen. Si la imagen es una plantilla, el contenedor es la “máquina virtual pequeña” que está corriendo. En esta práctica se crean **dos contenedores** a partir de la *misma* imagen `postgres:16-alpine`, cada uno con su propio nombre, su propia base de datos y su propio puerto.

¿Qué es un “volumen”??

Un **volumen** es un espacio de disco que Docker administra para que los datos del contenedor *sobrevivan* aunque el contenedor se elimine. Sin volumen, cuando se hace `docker compose down` se pierden todos los datos. En esta práctica se define un volumen por sede: `vol-cuenca` y `vol-quito`.

¿Qué es `docker-compose.yml`??

Es un archivo de texto plano con extensión `.yaml` (formato YAML) donde se declara *qué contenedores levantar juntos, con qué imagen, qué puertos, qué volúmenes y qué red compartir*. Docker lee este archivo y crea todo con un solo comando. YAML es sensible a la indentación: dos espacios por nivel, sin tabuladores; si mezcla espacios y tabuladores, Docker rechaza el archivo [8].

5.2 Crear el archivo `docker-compose.yml`

Paso 4.1 — Abrir VS Code (o Notepad++) en la carpeta del proyecto

Abra VS Code. Vaya al menú *File* → *Open Folder...* y seleccione `C:\uteq\banco-austro`. En el panel izquierdo (*Explorer*) deben aparecer las dos subcarpetas `sql-cuenca` y `sql-quito`. Pulse el botón de nuevo archivo (*New File*), llame al archivo exactamente `docker-compose.yml` y pulse *Enter*.

Si algo falla...

Windows a veces oculta las extensiones. Si al guardar aparece `docker-compose.yml.txt` entonces Windows añadió `.txt`. Para evitarlo: en el Explorador de Windows abra *Vista* → marque *Extensiones de archivos*. Así podrá verificar la extensión real y renombrar si hace falta.

Paso 4.2 — Escribir el contenido del `docker-compose.yml`

Copie el siguiente contenido exactamente. Respete los espacios (dos espacios por nivel de indentación) y no use tabuladores.

Listing 4: C:\uteq\banco-austro\docker-compose.yml

```

1 services:
2
3   db-cuenca:
4     image: postgres:16-alpine
5     container_name: banco-cuenca
6     environment:
7       POSTGRES_DB: banco_cuenca
8       POSTGRES_USER: banco
9       POSTGRES_PASSWORD: banco123
10    ports:
11      - "5432:5432"
12    volumes:
13      - vol-cuenca:/var/lib/postgresql/data
14      - ./sql-cuenca:/docker-entrypoint-initdb.d
15    networks:
16      - bda-net
17
18   db-quito:
19     image: postgres:16-alpine
20     container_name: banco-quito
21     environment:
22       POSTGRES_DB: banco_quito
23       POSTGRES_USER: banco
24       POSTGRES_PASSWORD: banco123
25    ports:
26      - "5433:5432"
27    volumes:
28      - vol-quito:/var/lib/postgresql/data
29      - ./sql-quito:/docker-entrypoint-initdb.d
30    networks:
31      - bda-net
32
33   networks:
34     bda-net:
35       driver: bridge
36
37   volumes:
38     vol-cuenca:
39     vol-quito:

```

5.3 Línea por línea: qué significa cada bloque

Línea o bloque	Explicación
<code>services:</code>	Bloque donde se declaran los contenedores. Cada entrada bajo <code>services</code> es un contenedor.
<code>db-cuenca:</code>	Nombre lógico del primer servicio. Puede llamarse como quiera; se usa para referenciarlo internamente.
<code>image: postgres:16-alpine</code>	Imagen que Docker descargará de Docker Hub la primera vez.
<code>container_name:</code>	Nombre real del contenedor. Aparece así en <code>docker ps</code> y en Docker Desktop.
<code>environment:</code>	Variables de entorno que la imagen oficial de PostgreSQL usa para inicializarse: nombre de la base, usuario y contraseña [4].
<code>ports:</code>	Mapeo puerto host → puerto contenedor. El motor escucha internamente en 5432; Cuenca se expone en 5432 y Quito en 5433 para no chocar entre sí.

<code>volumes:</code> (primera línea)	Los datos que PostgreSQL escribe en <code>/var/lib/postgresql/data</code> se persisten en el volumen <code>vol-cuenca</code> .
<code>volumes:</code> (segunda línea)	Monta la carpeta local <code>sql-cuenca</code> dentro de la carpeta especial <code>docker-entrypoint-initdb.d</code> del contenedor. <i>Cualquier archivo <code>.sql</code> que quede allí se ejecuta automáticamente al primer arranque de la base [4].</i>
<code>networks:</code> <code>bda-net</code>	Ambos contenedores se conectan a la misma red Docker; podrían resolverse por nombre entre sí (aunque en esta práctica el acceso será siempre desde el host).

6 Crear las tablas y datos de prueba

6.1 Repaso rápido de conceptos

¿Qué es un script “DDL”??

DDL significa *Data Definition Language*. Son las sentencias SQL que *definen la estructura*: CREATE TABLE, CREATE INDEX, ALTER TABLE. No manejan datos, solo describen el esquema.

¿Qué es un script “DML”??

DML significa *Data Manipulation Language*. Son las sentencias que *trabajan con datos*: INSERT, UPDATE, DELETE, SELECT. En esta práctica el DDL crea las tablas vacías y el DML inserta las filas iniciales.

¿Qué es el mecanismo docker-entrypoint-initdb.d??

Es una carpeta especial dentro de la imagen oficial de PostgreSQL. Al *primer* arranque de un contenedor, PostgreSQL revisa esa carpeta y ejecuta todos los archivos `.sql` que encuentre, en orden alfabético. Por eso se van a nombrar los scripts con prefijo numérico: `01_schema.sql` se ejecuta antes que `02_datos.sql`. En arranques posteriores esta ejecución **no se repite**: solo pasa la primera vez, cuando el volumen de datos está vacío [4].

6.2 Crear los scripts del nodo Cuenca

Paso 5.1 — Crear 01_schema.sql de Cuenca

En VS Code, con la carpeta `banco-austro` abierta, haga clic derecho sobre la carpeta `sql-cuenca` y elija *New File*. Nombre el archivo exactamente `01_schema.sql` y presione *Enter*. Copie el siguiente contenido:

Listing 5: `sql-cuenca/01_schema.sql`

```

1  -- Esquema del nodo Cuenca (sede central)
2  -- Fragmento horizontal: filas con oficina = 'CUENCA'
3
4  CREATE TABLE IF NOT EXISTS clientes (
5      id          BIGSERIAL PRIMARY KEY,
6      cedula      VARCHAR(10) UNIQUE NOT NULL,
7      nombre      VARCHAR(120) NOT NULL,
8      ciudad      VARCHAR(60)  NOT NULL,
9      creado_en   TIMESTAMP    NOT NULL DEFAULT NOW()
10 );
11
12 CREATE TABLE IF NOT EXISTS cuentas (
13     id          BIGSERIAL PRIMARY KEY,
14     numero      VARCHAR(20) UNIQUE NOT NULL,
15     cliente_id  BIGINT      NOT NULL REFERENCES clientes(id),
16     saldo       NUMERIC(15,2) NOT NULL DEFAULT 0.00
17               CHECK (saldo >= 0),
18     oficina     VARCHAR(20) NOT NULL CHECK (oficina = 'CUENCA'),
19     abierta_en  TIMESTAMP    NOT NULL DEFAULT NOW()
20 );
21
22 CREATE TABLE IF NOT EXISTS transacciones (
23     id          BIGSERIAL PRIMARY KEY,
24     cuenta_orig VARCHAR(20) NOT NULL,
25     cuenta_dest VARCHAR(20) NOT NULL,
26     monto       NUMERIC(15,2) NOT NULL CHECK (monto > 0),

```

```

27     fecha          TIMESTAMP NOT NULL DEFAULT NOW(),
28     estado         VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA',
29     oficina        VARCHAR(20) NOT NULL CHECK (oficina = 'CUENCA')
30 );
31
32 CREATE INDEX idx_cuentas_cliente ON cuentas(cliente_id);
33 CREATE INDEX idx_transacciones_orig ON transacciones(cuenta_orig);
34 CREATE INDEX idx_transacciones_fecha ON transacciones(fecha);

```

Nota técnica

Fije la atención en la restricción `CHECK (oficina = 'CUENCA')`. Ésta es la traducción a SQL del *predicado del fragmento horizontal*. Si más adelante un desarrollador intenta insertar una fila con `oficina = 'QUITO'` en el nodo Cuenca, PostgreSQL la rechaza automáticamente y la regla de disyunción de la fragmentación queda protegida a nivel de motor [2, cap. 3].

Paso 5.2 — Crear 02_datos.sql de Cuenca

En la misma carpeta `sql-cuenca`, cree un nuevo archivo llamado `02_datos.sql` y pegue lo siguiente:

Listing 6: sql-cuenca/02_datos.sql

```

1  -- Datos iniciales del nodo Cuenca
2
3  INSERT INTO clientes (cedula, nombre, ciudad) VALUES
4      ('0101234567', 'Mario Vintimilla', 'Cuenca'),
5      ('0107654321', 'Sofia Vasquez', 'Cuenca'),
6      ('0103456789', 'Andres Cordero', 'Cuenca');
7
8  INSERT INTO cuentas (numero, cliente_id, saldo, oficina) VALUES
9      ('2201000001', 1, 15000.00, 'CUENCA'),
10     ('2201000002', 2, 4200.50, 'CUENCA'),
11     ('2201000003', 3, 850.75, 'CUENCA');
12
13  INSERT INTO transacciones (cuenta_orig, cuenta_dest, monto, oficina) VALUES
14     ('2201000001', '2201000002', 500.00, 'CUENCA'),
15     ('2201000002', '2201000003', 120.25, 'CUENCA');

```

6.3 Crear los scripts del nodo Quito

Paso 5.3 — Scripts del nodo Quito

De la misma forma, en la carpeta `sql-quito` cree los archivos `01_schema.sql` y `02_datos.sql`. El esquema es idéntico al de Cuenca salvo que las restricciones `CHECK` referencian a `'QUITO'`; los datos son otros.

Listing 7: sql-quito/01_schema.sql

```

1  -- Esquema del nodo Quito
2  -- Fragmento horizontal: filas con oficina = 'QUITO'
3
4  CREATE TABLE IF NOT EXISTS clientes (
5      id          BIGSERIAL PRIMARY KEY,
6      cedula      VARCHAR(10) UNIQUE NOT NULL,
7      nombre      VARCHAR(120) NOT NULL,
8      ciudad      VARCHAR(60) NOT NULL,
9      creado_en   TIMESTAMP NOT NULL DEFAULT NOW()
10 );

```

```

11
12 CREATE TABLE IF NOT EXISTS cuentas (
13     id          BIGSERIAL PRIMARY KEY,
14     numero      VARCHAR(20) UNIQUE NOT NULL,
15     cliente_id  BIGINT    NOT NULL REFERENCES clientes(id),
16     saldo       NUMERIC(15,2) NOT NULL DEFAULT 0.00
17     CHECK (saldo >= 0),
18     oficina     VARCHAR(20) NOT NULL CHECK (oficina = 'QUITO'),
19     abierta_en  TIMESTAMP  NOT NULL DEFAULT NOW()
20 );
21
22 CREATE TABLE IF NOT EXISTS transacciones (
23     id          BIGSERIAL PRIMARY KEY,
24     cuenta_orig VARCHAR(20) NOT NULL,
25     cuenta_dest VARCHAR(20) NOT NULL,
26     monto       NUMERIC(15,2) NOT NULL CHECK (monto > 0),
27     fecha       TIMESTAMP  NOT NULL DEFAULT NOW(),
28     estado      VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA',
29     oficina     VARCHAR(20) NOT NULL CHECK (oficina = 'QUITO')
30 );
31
32 CREATE INDEX idx_cuentas_cliente    ON cuentas(cliente_id);
33 CREATE INDEX idx_transacciones_orig ON transacciones(cuenta_orig);
34 CREATE INDEX idx_transacciones_fecha ON transacciones(fecha);

```

Listing 8: sql-quito/02_datos.sql

```

1  -- Datos iniciales del nodo Quito
2
3  INSERT INTO clientes (cedula, nombre, ciudad) VALUES
4      ('1701234567', 'Pedro Jimenez', 'Quito'),
5      ('1709876543', 'Luisa Paredes', 'Quito'),
6      ('1704567890', 'Carla Espinoza', 'Quito');
7
8  INSERT INTO cuentas (numero, cliente_id, saldo, oficina) VALUES
9      ('1701000001', 1, 8000.00, 'QUITO'),
10     ('1701000002', 2, 2500.00, 'QUITO'),
11     ('1701000003', 3, 975.40, 'QUITO');
12
13  INSERT INTO transacciones (cuenta_orig, cuenta_dest, monto, oficina) VALUES
14     ('1701000001', '1701000002', 300.00, 'QUITO'),
15     ('1701000003', '1701000001', 75.00, 'QUITO');

```

6.4 Estructura del proyecto en este punto

Listing 9: C:\uteq\banco-austro tras crear todos los scripts

```

C:\uteq\banco-austro\
+-- docker-compose.yml
+-- sql-cuenca\
| +-- 01_schema.sql
| +-- 02_datos.sql
+-- sql-quito\
    +-- 01_schema.sql
    +-- 02_datos.sql

```

7 Levantar los contenedores por primera vez

Paso 6.1 — Asegurarse de que Docker Desktop está corriendo

Abra Docker Desktop desde el menú de Inicio. Espere a que el ícono en la barra de tareas quede en verde (o a que la ventana muestre “Docker Desktop is running”). Sin Docker Desktop corriendo, todos los comandos siguientes fallan con **error during connect**.

Paso 6.2 — Ejecutar `docker compose up`

En `cmd.exe`, cambie a la carpeta del proyecto y levante los servicios:

```
cd C:\uteq\banco-austro
docker compose up -d
```

La primera vez, Docker descarga la imagen `postgres:16-alpine` (unos 250 MB). Espere el mensaje similar a:

```
[+] Running 5/5
- Network banco-austro_bda-net Created
- Volume "banco-austro_vol-cuenca" Created
- Volume "banco-austro_vol-quito" Created
- Container banco-cuenca Started
- Container banco-quito Started
```

¿Qué es la bandera `-d`??

`-d` significa *detached*: Docker levanta los servicios *en segundo plano* y le devuelve el control de la consola. Sin `-d`, la consola queda ocupada mostrando los logs de los contenedores y hay que abrir otra ventana para trabajar.

Paso 6.3 — Verificar los contenedores en la línea de comandos

Todavía en `cmd.exe`, ejecute:

```
docker compose ps
```

Debe listar los dos contenedores en estado **running** y con los puertos `0.0.0.0:5432->5432/tcp` y `0.0.0.0:5433->5433/tcp`.

Paso 6.4 — Verificar los logs de inicialización

Muestre las últimas líneas del log de cada contenedor:

```
docker logs banco-cuenca --tail 30
docker logs banco-quito --tail 30
```

En cada uno debe aparecer que los scripts se ejecutaron y que la base está lista.

¿Cómo saber si salió bien?

En los logs debe encontrar líneas parecidas a las siguientes:

```
running /docker-entrypoint-initdb.d/01_schema.sql
running /docker-entrypoint-initdb.d/02_datos.sql
database system is ready to accept connections
```

Si esas tres líneas aparecen, los scripts SQL se ejecutaron correctamente y el motor está en pie escuchando. Repita para ambos contenedores.

Paso 6.5 — Verificar los contenedores en Docker Desktop (interfaz gráfica)

Abra la ventana de Docker Desktop. En el menú lateral izquierdo, seleccione *Containers*. Debe ver un grupo llamado **banco-austro** con dos entradas: **banco-cuenca** y **banco-quito**, ambas con estado *Running* (punto verde).

Haga clic en cualquiera de los dos contenedores y luego en la pestaña *Logs* para inspeccionar visualmente lo mismo que se vio con `docker logs`. La pestaña *Terminal* le da una *shell* dentro del contenedor si necesita entrar manualmente [8].

Si algo falla...

Error Bind for 0.0.0.0:5432 failed: port is already allocated. El puerto 5432 ya está en uso, seguramente porque tiene una instalación local de PostgreSQL corriendo. Dos alternativas: (a) detener el servicio local de PostgreSQL desde *Servicios de Windows*; o (b) editar `docker-compose.yml` y cambiar "5432:5432" por "5442:5432" y "5433:5432" por "5443:5432". Si toma la opción (b), recuerde luego usar 5442/5443 en los pasos siguientes.

Error yaml: line X: mapping values are not allowed here. Su archivo `docker-compose.yml` tiene un tabulador o una indentación inconsistente. Ábralo en VS Code y verifique que use exactamente dos espacios por nivel; ningún tabulador.

8 Verificar los datos con pgAdmin (opción gráfica)

Antes de escribir una sola línea de código Java conviene comprobar, con una herramienta gráfica de bases de datos, que los dos motores contienen exactamente el fragmento que deberían. Se muestran dos alternativas equivalentes: pgAdmin 4 (herramienta dedicada) o el panel Database de IntelliJ IDEA (integrado en el IDE).

8.1 Configurar pgAdmin 4

Paso 7.1 — Abrir pgAdmin y crear dos servidores

Abra pgAdmin 4. Se abrirá en su navegador (<http://127.0.0.1:PUERTO>). En el panel izquierdo, haga clic derecho sobre *Servers* → *Register* → *Server...*. Complete la primera conexión con los siguientes datos y pulse *Save*:

Pestaña General	Name: Banco Austro - Cuenca
Pestaña Connection	Host name/address: localhost
	Port: 5432
	Maintenance database: banco_cuenca
	Username: banco
	Password: banco123
	Save password: marcado

Repita el mismo proceso para el segundo servidor, con nombre Banco Austro - Quito, puerto 5433 y base banco_quito.

Paso 7.2 — Navegar hasta las tablas

En el panel izquierdo, expanda *Servers* → *Banco Austro - Cuenca* → *Databases* → *banco_cuenca* → *Schemas* → *public* → *Tables*. Debe ver tres tablas: *clientes*, *cuentas* y *transacciones*. Repita en el servidor de Quito.

Paso 7.3 — Consultar los fragmentos

En pgAdmin, con el servidor de Cuenca seleccionado, abra el editor de consultas: botón *Query Tool* (ícono con un rayo). Escriba y ejecute (botón de *play*, o tecla *F5*):

```

1 SELECT c.numero, cl.nombre, c.saldo, c.oficina
2 FROM cuentas c
3 JOIN clientes cl ON cl.id = c.cliente_id
4 ORDER BY c.numero;
```

Debe obtener tres filas: las cuentas 2201000001, 2201000002 y 2201000003, con sus titulares. Cambie al servidor de Quito y ejecute la misma consulta: obtendrá otras tres filas, las que empiezan con 17.

¿Cómo saber si salió bien?

La *regla de disjunción* de la fragmentación se verifica empíricamente cuando se comprueba que ninguna cuenta está en los dos nodos al mismo tiempo. La *regla de completitud* se verifica sumando: 3 filas en Cuenca + 3 filas en Quito = 6 filas en la relación lógica completa. Esta comprobación numérica corresponde al teorema de reconstrucción de Özsu y Valduriez [2, cap. 3].

8.2 Alternativa: panel Database de IntelliJ IDEA

Paso 7.4 — Panel Database de IntelliJ

En IntelliJ IDEA, abra la vista *Database* desde *View* → *Tool Windows* → *Database*. Pulse el ícono *+* → *Data Source* → *PostgreSQL* y configure dos fuentes con los mismos parámetros de la tabla anterior. Pulse *Test Connection*; deben responder *Successful*. Si es la primera vez, IntelliJ le pedirá descargar el *driver JDBC*; acepte y espere unos segundos.

Desde aquí puede ejecutar las mismas consultas de verificación: botón derecho sobre la conexión → *New* → *Query Console*.

9 Crear el proyecto Spring Boot

9.1 Repaso rápido de conceptos

¿Qué es Spring Boot??

Spring Boot es un *framework* de Java que se encarga de la parte tediosa de arrancar una aplicación de servidor: configura un servidor web embebido (Tomcat), lee la configuración, conecta a bases de datos, expone endpoints REST y publica métricas, sin que usted tenga que escribirlo. En esta práctica se usa Spring Boot 3.x sobre Java 21 (LTS) [9].

¿Qué es Spring Initializr??

Es un generador oficial de proyectos Spring Boot. Recibe los parámetros (Java, Maven, dependencias) y devuelve un ZIP con la estructura completa. IntelliJ IDEA lo tiene integrado en el asistente *New Project*.

¿Qué es un “DataSource”??

Un **DataSource** es el objeto Java que representa una conexión (o pool de conexiones) a una base de datos. En esta práctica se necesitan **dos DataSource**: uno hacia **banco_cuenca** y otro hacia **banco_quito**. Éste es el punto exacto donde se materializa la “distribución” en el código cliente.

9.2 Generar el proyecto

Paso 8.1 — New Project en IntelliJ

Abra IntelliJ IDEA. En la pantalla de bienvenida pulse *New Project*. En el panel izquierdo elija *Spring Boot* (o *Spring Initializr* según la versión). Complete el asistente con:

- Name: `banco-austro`
- Location: `C:\uteq\banco-austro`
- Language: Java
- Type: Maven
- Group: `ec.edu.uteq`
- Artifact: `banco-austro`
- Package name: `ec.edu.uteq.bancoaustro`
- JDK: 21 (Temurin)
- Java: 21
- Packaging: Jar

Pulse *Next*.

Paso 8.2 — Elegir Spring Boot y dependencias

En la siguiente pantalla:

- Spring Boot: la última estable 3.3.x o 3.4.x
- Dependencies: marque estas tres:
 - *Spring Web* (para exponer endpoints REST)
 - *JDBC API* (para conectarse a la base con `JdbcTemplate`)
 - *PostgreSQL Driver* (para que Java hable con PostgreSQL)

Pulse *Create*. IntelliJ descargará las dependencias de Maven y abrirá el proyecto. Espere a que el indicador de fondo diga que la indexación terminó.

9.3 Estructura del proyecto que se genera

Listing 10: Estructura Maven típica de Spring Boot

```
banco-austro\
+-- pom.xml (dependencias Maven)
+-- src\
| +-- main\
| | +-- java\ec\edu\uteq\bancoaustro\
| | | +-- BancoAustroApplication.java (clase principal)
| | +-- resources\
| | +-- application.properties (configuracion)
| +-- test\
| +-- java\ec\edu\uteq\bancoaustro\
| +-- BancoAustroApplicationTests.java
+-- (Docker files ya existentes)
+-- (sql-cuenca, sql-quito ya existentes)
```

Nota técnica

Es importante que el proyecto se cree *dentro* de `C:\uteq\banco-austro`, junto al `docker-compose.yml` y las carpetas `sql-cuenca` y `sql-quito`. Así toda la práctica queda autocontenida.

9.4 Cambiar `application.properties` por `application.yml`

Por comodidad de lectura y consistencia con lo que hicimos con Docker, se convierte el archivo de configuración al formato YAML.

Paso 8.3 — Renombrar y editar

En el panel de proyecto, expanda `src/main/resources`. Clic derecho sobre `application.properties` → *Refactor* → *Rename* → póngale de nombre `application.yml`. Reemplace su contenido por:

Listing 11: `src/main/resources/application.yml`

```
1 server:
2   port: 8080
3
4 spring:
5   application:
6     name: banco-austro
7
```

```
8  datasources:
9    cuenca:
10      url: jdbc:postgresql://localhost:5432/banco_cuenca
11      username: banco
12      password: banco123
13      driver-class-name: org.postgresql.Driver
14    quito:
15      url: jdbc:postgresql://localhost:5433/banco_quito
16      username: banco
17      password: banco123
18      driver-class-name: org.postgresql.Driver
```

¿Qué es cada línea de esta configuración??

server.port: 8080 indica que la aplicación Spring Boot escuchará peticiones HTTP en el puerto 8080 de la máquina local. La sección **datasources.cuenca** declara *dónde vive* el nodo Cuenca (**localhost:5432**) y qué credenciales usar; **datasources.quito** hace lo propio para Quito. **jdbc:postgresql://...** es la sintaxis estándar de una URL JDBC para PostgreSQL [4].

10 Escribir el código Java de la aplicación

10.1 Organizar los paquetes

Se van a crear tres paquetes dentro de `ec.edu.uteq.bancoaustro`:

- **config**: contiene la clase que declara los dos `DataSource`.
- **service**: contiene la lógica de enrutamiento distribuido.
- **controller**: contiene los endpoints REST.

Paso 9.1 — Crear los tres paquetes

En el panel del proyecto, expanda `src/main/java/ec/edu/uteq/bancoaustro`. Clic derecho sobre el paquete `bancoaustro` → *New* → *Package*, y escriba `config`. Repita para `service` y para `controller`.

10.2 Clase de configuración de dos DataSource

Paso 9.2 — Crear DataSourceConfig.java

Dentro del paquete `config`, clic derecho → *New* → *Java Class* → nombre `DataSourceConfig`. Reemplace el contenido por:

Listing 12: `config/DataSourceConfig.java`

```

1 package ec.edu.uteq.bancoaustro.config;
2
3 import javax.sql.DataSource;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.boot.jdbc.DataSourceBuilder;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8
9 @Configuration
10 public class DataSourceConfig {
11
12     @Value("${datasources.cuenca.url}")
13     private String cuencaUrl;
14     @Value("${datasources.cuenca.username}")
15     private String cuencaUser;
16     @Value("${datasources.cuenca.password}")
17     private String cuencaPass;
18
19     @Value("${datasources.quito.url}")
20     private String quitoUrl;
21     @Value("${datasources.quito.username}")
22     private String quitoUser;
23     @Value("${datasources.quito.password}")
24     private String quitoPass;
25
26     @Bean(name = "dsCuenca")
27     public DataSource dsCuenca() {
28         return DataSourceBuilder.create()
29             .url(cuencaUrl)
30             .username(cuencaUser)
31             .password(cuencaPass)
32             .driverClassName("org.postgresql.Driver")
33             .build();
34     }
35
36     @Bean(name = "dsQuito")

```

```

37     public DataSource dsQuito() {
38         return DataSourceBuilder.create()
39             .url(quitoUrl)
40             .username(quitoUser)
41             .password(quitoPass)
42             .driverClassName("org.postgresql.Driver")
43             .build();
44     }
45 }

```

¿Qué es esta clase, en simple??

La anotación `@Configuration` le dice a Spring que la clase declara objetos gestionados. Cada método con `@Bean` le entrega un objeto a Spring para que lo use cuando alguien pida *ese* tipo. En este caso se crean dos `DataSource` y se identifican con nombres distintos (`"dsCuenca"` y `"dsQuito"`) para poder pedirlos por su nombre en otras clases con `@Qualifier` [9].

10.3 Servicio de consulta distribuida

Paso 9.3 — Crear `ConsultaDistribuidaService.java`

En el paquete `service`, cree la clase `ConsultaDistribuidaService` con este contenido:

Listing 13: `service/ConsultaDistribuidaService.java`

```

1  package ec.edu.uteq.bancoaustro.service;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Map;
6  import javax.sql.DataSource;
7  import org.springframework.beans.factory.annotation.Qualifier;
8  import org.springframework.jdbc.core.JdbcTemplate;
9  import org.springframework.stereotype.Service;
10
11 @Service
12 public class ConsultaDistribuidaService {
13
14     private final JdbcTemplate jdbcCuenca;
15     private final JdbcTemplate jdbcQuito;
16
17     public ConsultaDistribuidaService(
18         @Qualifier("dsCuenca") DataSource dsCuenca,
19         @Qualifier("dsQuito") DataSource dsQuito) {
20         this.jdbcCuenca = new JdbcTemplate(dsCuenca);
21         this.jdbcQuito = new JdbcTemplate(dsQuito);
22     }
23
24     public Map<String, Object> consultarSaldo(String numero) {
25         JdbcTemplate destino = enrutar(numero);
26         String sql = "SELECT numero, saldo, oficina "
27             + "FROM cuentas WHERE numero = ?";
28         List<Map<String, Object>> filas =
29             destino.queryForList(sql, numero);
30         if (filas.isEmpty()) {
31             return Map.of(
32                 "error", "Cuenta no encontrada",
33                 "numero", numero);
34         }
35         return filas.get(0);

```



```

36     }
37
38     public List<Map<String, Object>> listarTodosLosClientes() {
39         String sql = "SELECT cedula, nombre, ciudad FROM clientes";
40         List<Map<String, Object>> union = new ArrayList<>();
41         union.addAll(jdbcCuenca.queryForList(sql));
42         union.addAll(jdbcQuito.queryForList(sql));
43         return union;
44     }
45
46     private JdbcTemplate enrutar(String numero) {
47         if (numero == null || numero.length() < 2) {
48             throw new IllegalArgumentException(
49                 "Numero de cuenta invalido: " + numero);
50         }
51         String prefijo = numero.substring(0, 2);
52         return switch (prefijo) {
53             case "22" -> jdbcCuenca;
54             case "17" -> jdbcQuito;
55             default -> throw new IllegalArgumentException(
56                 "Prefijo de oficina no reconocido: " + prefijo);
57         };
58     }
59 }

```

¿Qué es esta clase, en simple??

El método `enrutar` mira los dos primeros dígitos del número de cuenta y decide en qué nodo consultar; ésta es la implementación de la *transparencia de fragmentación*: el cliente HTTP nunca sabe cuántos nodos hay [2, cap. 1]. El método `listarTodosLosClientes` ilustra la *operación de unión* sobre fragmentos: se hace `SELECT` en los dos nodos y se concatenan las filas. Como `clientes` está replicada, en producción esta unión produciría duplicados y sería necesario un `DISTINCT` o consultar solo a un nodo; se deja así intencionalmente para exponer la discusión en clase.

10.4 Controlador REST

Paso 9.4 — Crear BancoController.java

En el paquete `controller`, cree la clase `BancoController` con este contenido:

Listing 14: controller/BancoController.java

```

1 package ec.edu.uteq.bancoaustro.controller;
2
3 import java.util.List;
4 import java.util.Map;
5 import ec.edu.uteq.bancoaustro.service.ConsultaDistribuidaService;
6 import org.springframework.web.bind.annotation.GetMapping;
7 import org.springframework.web.bind.annotation.PathVariable;
8 import org.springframework.web.bind.annotation.RequestMapping;
9 import org.springframework.web.bind.annotation.RestController;
10
11 @RestController
12 @RequestMapping("/api/banco")
13 public class BancoController {
14
15     private final ConsultaDistribuidaService service;
16
17     public BancoController(ConsultaDistribuidaService service) {

```

```
18         this.service = service;
19     }
20
21     @GetMapping("/saldo/{numero}")
22     public Map<String, Object> saldo(@PathVariable String numero) {
23         return service.consultarSaldo(numero);
24     }
25
26     @GetMapping("/clientes")
27     public List<Map<String, Object>> clientes() {
28         return service.listarTodosLosClientes();
29     }
30 }
```

10.5 Estructura del proyecto Java en este punto

Listing 15: Código del proyecto Spring Boot

```
src\main\java\ec\edu\uteq\bancoaustrro\
+-- BancoAustroApplication.java (generado por Spring Initializr)
+-- config\
| +-- DataSourceConfig.java
+-- service\
| +-- ConsultaDistribuidaService.java
+-- controller\
    +-- BancoController.java
```

11 Ejecutar y probar la aplicación

11.1 Levantar la aplicación Spring Boot

Paso 10.1 — Ejecutar desde IntelliJ

En el panel del proyecto, abra la clase `BancoAustroApplication.java`. Haga clic en el triángulo verde que aparece a la izquierda del nombre de la clase, o al lado del método `main`, y elija *Run 'BancoAustroApplication'*. También funciona el atajo *Shift+F10*.

En la parte inferior aparecerá la consola de ejecución con logs de Spring Boot. Espere a la línea:

```
Tomcat started on port(s): 8080 (http)
Started BancoAustroApplication in X.XXX seconds
```

Mientras esa línea no aparezca, la aplicación todavía no está lista para recibir peticiones.

Si algo falla...

Fallo de conexión `Connection to localhost:5432 refused`. La aplicación Java no puede alcanzar al motor Cuenca. Compruebe con `docker compose ps` que ambos contenedores están corriendo. Si no lo están, levante con `docker compose up -d`.

`Could not resolve placeholder 'datasources.cuenca.url'`. El archivo `application.yml` tiene un error de indentación o está mal nombrado. Verifique el nombre exacto y que la indentación use dos espacios por nivel.

Error de compilación en switch. La sintaxis `switch (...) ->` requiere Java 14 o superior; verifique en *File* → *Project Structure* que el proyecto está configurado con SDK 21 y *Language level 21*.

11.2 Probar los endpoints con Postman

Paso 10.2 — Primera petición GET en Postman

Abra Postman. En la parte superior de la pestaña sin título, seleccione el método `GET` en el desplegable de la izquierda y escriba la URL:

```
http://localhost:8080/api/banco/saldo/2201000001
```

Pulse el botón azul *Send*. En el panel inferior de la respuesta debe recibir un JSON similar a:

```
{
  "numero": "2201000001",
  "saldo": 15000.00,
  "oficina": "CUENCA"
}
```

La cuenta empieza con 22, entonces la aplicación la buscó en el nodo Cuenca. Si el JSON responde `.oficina: CUENCA`, la ruta fue correcta.

Paso 10.3 — Segunda petición: consultar el nodo Quito

Cambie la URL a:

```
http://localhost:8080/api/banco/saldo/1701000001
```

Pulse *Send*. Ahora la respuesta debe ser:

```
{
  "numero": "1701000001",
  "saldo": 8000.00,
  "oficina": "QUITO"
}
```

```
}
```

El prefijo 17 enrutó al nodo Quito.

Paso 10.4 — Tercera petición: unión de los dos nodos

Cambie la URL a:

```
http://localhost:8080/api/banco/clientes
```

Debe recibir un arreglo JSON con **seis** entradas: los tres clientes de Cuenca seguidos de los tres clientes de Quito. Ésta es la operación de *unión* sobre fragmentos.

11.3 Alternativa: probar con el navegador o con curl

Paso 10.5 — Con el navegador

Como las tres peticiones son GET públicas, también funcionan pegando la URL en la barra de direcciones de Chrome, Edge o Firefox. Los navegadores modernos formatean automáticamente el JSON de respuesta.

Paso 10.6 — Con curl.exe en cmd.exe

En Windows 10 + existe el ejecutable `curl.exe` preinstalado. Abra `cmd.exe` y ejecute:

```
curl.exe http://localhost:8080/api/banco/saldo/2201000001
curl.exe http://localhost:8080/api/banco/saldo/1701000001
curl.exe http://localhost:8080/api/banco/clientes
```

¿Cómo saber si salió bien?

La práctica es *correcta* cuando las tres peticiones responden *sin cambiar una sola línea de código cliente*, aunque los datos residan en dos motores físicamente separados. En eso consiste la **transparencia de ubicación** en una base de datos distribuida [2, cap. 1].

12 Prueba de tolerancia a fallos

Uno de los cinco requisitos del caso era: “si una oficina se cae, las demás siguen operando”. Se comprueba empíricamente.

Paso 11.1 — Detener el nodo Quito

Con la aplicación Spring Boot todavía corriendo, abra un nuevo `cmd.exe` y ejecute:

```
docker stop banco-quito
docker compose ps
```

El listado debe mostrar a `banco-cuenca` como `running` y a `banco-quito` como `exited`. En Docker Desktop, el punto verde de Quito pasará a gris.

Paso 11.2 — Verificar la degradación selectiva

Regrese a Postman y vuelva a lanzar las peticiones:

- `GET /api/banco/saldo/2201000001` → **sigue respondiendo correctamente** porque la cuenta está en Cuenca, que sigue viva.
- `GET /api/banco/saldo/1701000001` → **falla** con un error de conexión al puerto 5433. La aplicación no sabe cómo llegar a Quito.
- `GET /api/banco/clientes` → **falla** porque el método actual intenta consultar los dos nodos y no captura el error del nodo caído.

Nota técnica

Esta caída selectiva es exactamente lo que en una práctica posterior se resuelve con un *Circuit Breaker*: cuando un nodo falla, el patrón detecta la falla, “abre el circuito” y responde con un fallback (por ejemplo, “cuenta temporalmente inaccesible”) sin bloquear al usuario [10]. El requisito de que “las demás sedes sigan operando” se cumple: aunque `/clientes` falle, la consulta de saldos de cuentas de Cuenca sigue funcionando.

Paso 11.3 — Reactivar el nodo Quito

En `cmd.exe`:

```
docker start banco-quito
docker compose ps
```

Espere unos segundos hasta que Quito quede `running` nuevamente. Repita las tres peticiones en Postman: las tres deben responder como al inicio.

13 Detener y limpiar el ambiente

Al terminar el laboratorio conviene liberar recursos.

Paso 12.1 — Detener la aplicación Java

En IntelliJ, en la ventana *Run*, pulse el botón rojo del cuadrado (*Stop*). La aplicación Spring Boot deja de escuchar en el puerto 8080.

Paso 12.2 — Detener los contenedores conservando los datos

Desde `cmd.exe` en la carpeta del proyecto:

```
cd C:\uteq\banco-austro
docker compose down
```

Este comando *para y elimina* los contenedores, pero **conserva los volúmenes**. La próxima vez que ejecute `docker compose up -d`, los datos seguirán ahí (los scripts `01_schema.sql` y `02_datos.sql` *no* se vuelven a ejecutar).

Paso 12.3 — Limpieza completa (opcional)

Si quiere reiniciar todo desde cero para volver a probar los scripts:

```
docker compose down -v
```

La bandera `-v` elimina también los volúmenes: los datos se pierden y, al volver a levantar los contenedores, los scripts se ejecutarán de nuevo desde el principio.

14 Cierre: qué se logró y qué queda pendiente

14.1 Propiedades verificadas

Al completar los doce pasos, el prototipo evidencia estas propiedades clásicas de una BDD [2, 3]:

- **Transparencia de fragmentación:** el cliente HTTP interactúa con un endpoint único y desconoce cuántos nodos existen.
- **Transparencia de ubicación:** el número de cuenta no revela dónde vive el dato desde el punto de vista del usuario; solo el enrutador interno lo interpreta.
- **Fragmentación horizontal completa, reconstruible y disjunta:** garantizada por las restricciones CHECK (`oficina = ...`) en cada nodo y por la convención de prefijos.
- **Autonomía local:** cada nodo procesa sus consultas locales aunque el otro nodo esté caído.
- **Consistencia con degradación de disponibilidad:** coherente con el posicionamiento CP en el análisis CAP [11] y con el modelo PACELC [12].

14.2 Qué queda pendiente para próximas prácticas

Este prototipo es didáctico. En un banco real haría falta agregar, al menos:

- **Commit en dos fases (2PC)** para transferencias entre cuentas de distinta sede [5].
- **Circuit Breaker** en la aplicación Spring Boot para que la caída de un nodo no propague errores a los endpoints que no lo necesitan [10].
- **Replicación lógica** activa entre los dos motores para la tabla `clientes`, evitando la escritura duplicada manual [4].
- **Migración a un producto nativo distribuido** (CockroachDB, PostgreSQL + Citus, Google Spanner) para observar cómo el motor asume por sí mismo la fragmentación y el consenso [6, 7].
- **Reloj lógico** para ordenar transacciones cross-site sin depender del reloj de pared [1].

14.3 Ejercicios propuestos

1. Agregar el nodo **Guayaquil** con prefijo 09. Extender `docker-compose.yml`, crear la carpeta `sql-guayaquil`, extender `DataSourceConfig` y añadir el `case "09"` en el enrutador.
2. Implementar un endpoint `POST /api/banco/transferencia` que reciba `origen`, `destino` y `monto`, y que actualice el saldo en el nodo correspondiente. Manejar el caso de origen y destino en distinta sede.
3. Añadir un *fallback* al método `listarTodosLosClientes`: si un nodo está caído, retornar solo los clientes disponibles con una advertencia en la respuesta.
4. Discutir en clase: ¿por qué la tabla `clientes` está replicada y no fragmentada? ¿Qué cambiaría si Banco del Austro creciera a un millón de titulares?

14.4 Glosario final

Término	Definición operacional
Sede	Ubicación lógica de una copia del motor con parte del dato; en el prototipo se implementa una sede como un contenedor Docker.
Nodo	Instancia individual del motor de base de datos que participa en el sistema distribuido.
Fragmento	Subconjunto de filas o columnas de una tabla que reside en un nodo específico.
Réplica	Copia completa de una tabla presente en más de un nodo.
Contenedor	Instancia en ejecución de una imagen Docker.
Volumen	Espacio de disco que persiste datos más allá de la vida del contenedor.
Fragmentación horizontal	División de una tabla por filas según un predicado, como <code>oficina = 'CUENCA'</code> .
Transparencia de fragmentación	El cliente no necesita saber que la tabla está partida en varios nodos.
Transparencia de ubicación	El cliente no necesita saber en qué nodo vive cada dato.
Autonomía local	Capacidad de un nodo de seguir operando cuando otros nodos caen.
DDL	<i>Data Definition Language</i> : CREATE, ALTER, DROP.
DML	<i>Data Manipulation Language</i> : INSERT, UPDATE, DELETE, SELECT.

Referencias

- [1] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.
- [2] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham, Switzerland: Springer International Publishing, 2020.
- [3] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1st ed. Sebastopol, CA, USA: O’Reilly Media, 2017.
- [4] PostgreSQL Global Development Group, “PostgreSQL 16 documentation,” <https://www.postgresql.org/docs/16/>, 2024, accedido: 2026-07-09.
- [5] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Francisco, CA, USA: Morgan Kaufmann, 1993.
- [6] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. Maastricht, The Netherlands: Maarten van Steen, 2023.
- [7] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2012.
- [8] Docker, Inc., “Docker compose specification,” <https://docs.docker.com/compose/>, 2024, accedido: 2026-07-09.
- [9] VMware Tanzu / Broadcom, “Spring boot 3.x reference documentation,” <https://docs.spring.io/spring-boot/docs/current/reference/html/>, 2024, accedido: 2026-07-09.
- [10] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2021.

- [11] E. Brewer, “CAP twelve years later: How the “rules” have changed,” *Computer*, vol. 45, no. 2, pp. 23–29, Feb. 2012.
- [12] D. J. Abadi, “Consistency tradeoffs in modern distributed database system design: CAP is only part of the story,” *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012.